



MHH772 / 90DaysOfDevOps



Code Pull requests Agents Actions Projects Wiki Security and quality Insights

Settings



master

90DaysOfDevOps / 2026 / day-25



LondheShubham153 Added day 25 task

e126e84 · 7 months ago



This branch is 22 commits ahead of TrainWithShubham/90DaysOfDevOps:master



Contribute



Sync fork

Name

Name

Last commit date



..



README.md

Added day 25 task

7 months ago

README.md



# Day 25 – Git Reset vs Revert & Branching Strategies

## Task

You'll learn how to **undo mistakes** safely — one of the most important skills in Git. You'll also explore **branching strategies** used by real engineering teams to manage code at scale.

## Expected Output

- A markdown file: `day-25-notes.md` with your observations and answers
- Continue updating `git-commands.md` in your `devops-git-practice` repo

# Challenge Tasks

## Task 1: Git Reset — Hands-On

1. Make 3 commits in your practice repo (commit A, B, C)
2. Use `git reset --soft` to go back one commit — what happens to the changes?
3. Re-commit, then use `git reset --mixed` to go back one commit — what happens now?
4. Re-commit, then use `git reset --hard` to go back one commit — what happens this time?
5. Answer in your notes:
  - What is the difference between `--soft`, `--mixed`, and `--hard`?
  - Which one is destructive and why?
  - When would you use each one?
  - Should you ever use `git reset` on commits that are already pushed?

## Task 2: Git Revert — Hands-On

1. Make 3 commits (commit X, Y, Z)
2. Revert commit Y (the middle one) — what happens?
3. Check `git log` — is commit Y still in the history?
4. Answer in your notes:
  - How is `git revert` different from `git reset`?
  - Why is revert considered **safer** than reset for shared branches?
  - When would you use revert vs reset?

## Task 3: Reset vs Revert — Summary

Create a comparison in your notes:

|                                  | <code>git reset</code> | <code>git revert</code> |
|----------------------------------|------------------------|-------------------------|
| What it does                     | ?                      | ?                       |
| Removes commit from history?     | ?                      | ?                       |
| Safe for shared/pushed branches? | ?                      | ?                       |
| When to use                      | ?                      | ?                       |

## Task 4: Branching Strategies

Research the following branching strategies and document each in your notes with:

- How it works (short description)
  - A simple diagram or flow (text-based is fine)
  - When/where it's used
  - Pros and cons
1. **GitFlow** — develop, feature, release, hotfix branches
  2. **GitHub Flow** — simple, single main branch + feature branches
  3. **Trunk-Based Development** — everyone commits to main, short-lived branches
  4. Answer:
    - Which strategy would you use for a startup shipping fast?
    - Which strategy would you use for a large team with scheduled releases?
    - Which one does your favorite open-source project use? (check any repo on GitHub)

## Task 5: Git Commands Reference Update

Update your `git-commands.md` to cover everything from Days 22–25:

- Setup & Config
- Basic Workflow (add, commit, status, log, diff)
- Branching (branch, checkout, switch)
- Remote (push, pull, fetch, clone, fork)
- Merging & Rebasing
- Stash & Cherry Pick
- Reset & Revert

## Hints

---

- `git reflog` is your safety net — it shows everything Git has done, even after a hard reset
- For branching strategies, look at how projects like Kubernetes, React, or Linux kernel manage branches

## Submission

---

1. Add your `day-25-notes.md` to `2026/day-25/`
2. Update `git-commands.md` — commit and push
3. Push to your fork

## Learn in Public

---

Share your Reset vs Revert comparison or your branching strategy notes on LinkedIn.

`#90DaysOfDevOps` `#DevOpsKaJosh` `#TrainWithShubham`

Happy Learning! **TrainWithShubham**